The Networking Ecosystem

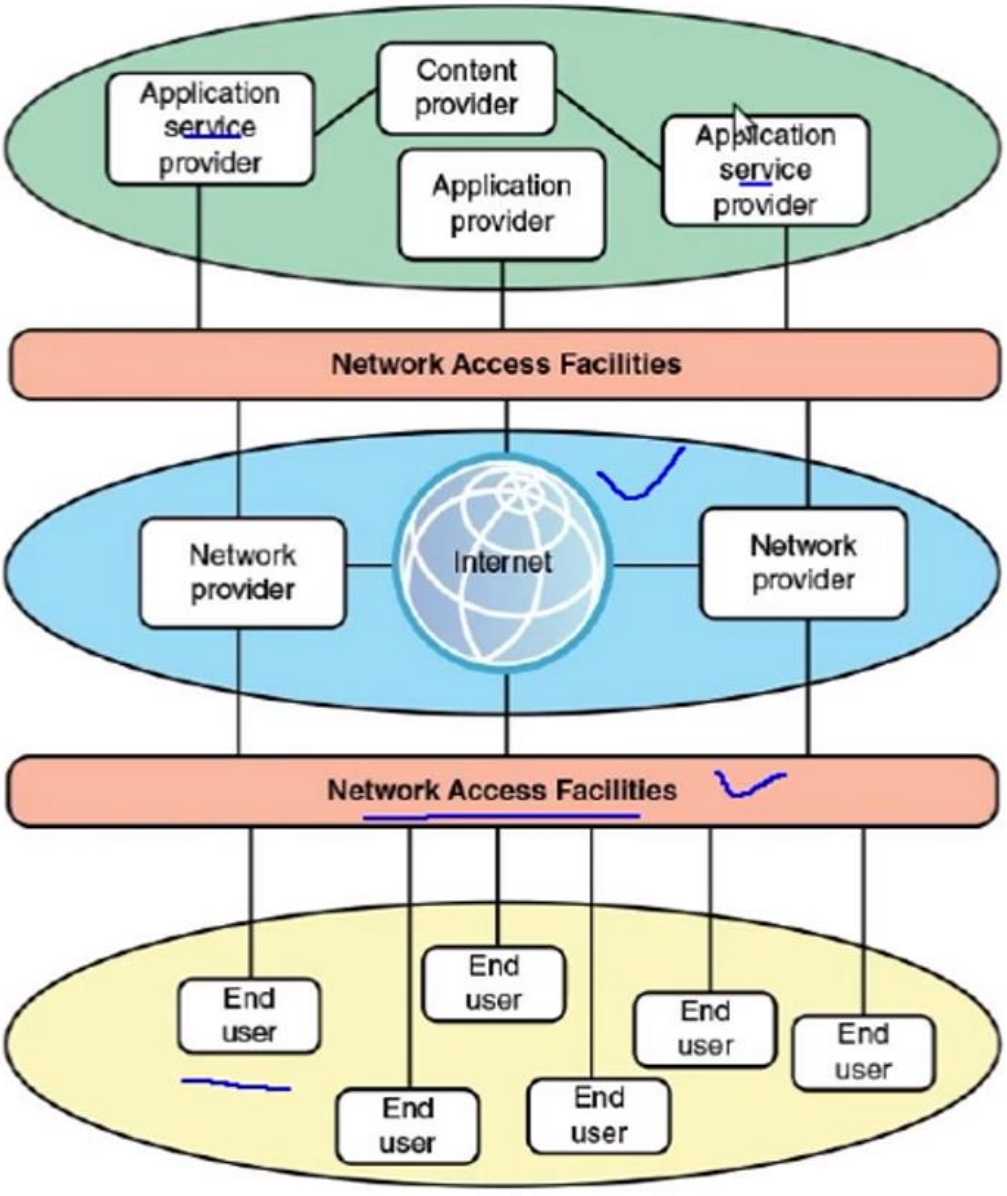


Figure: The Modern Networking Ecosystem

INTRODUCTION TO SDN

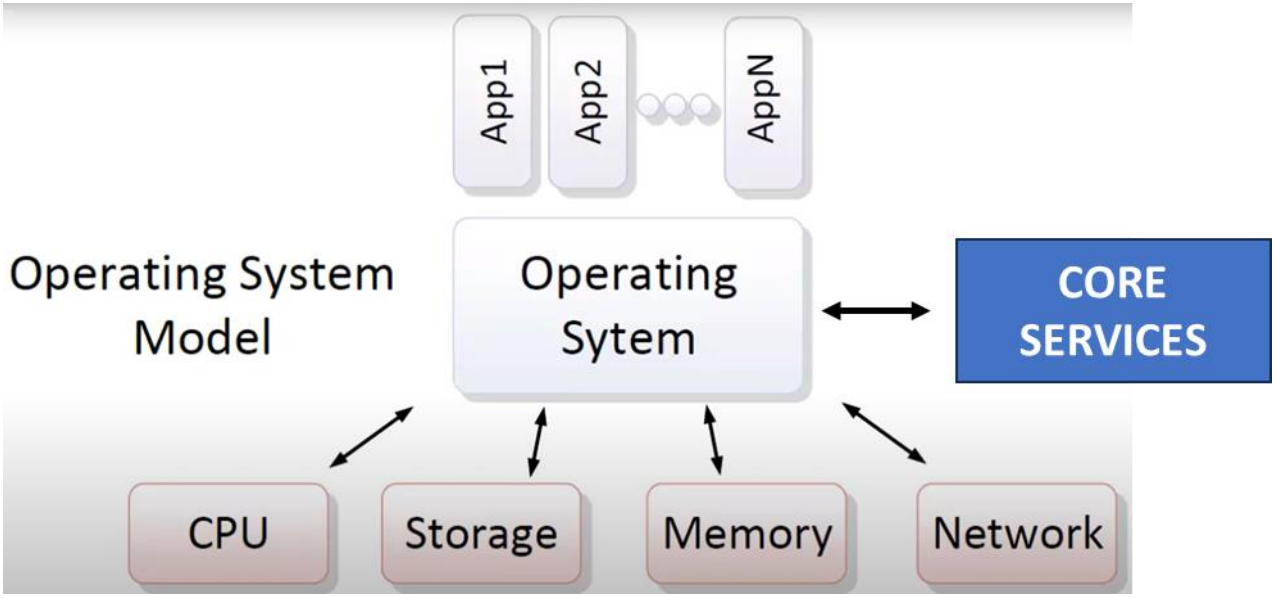
- Primary Goal of SDN
- 3 layer model of operating systems (analogy for an SDN model)
- 3 layer SDN model
- Layer details
- Packet walkthrough example
- Availability and scalability
- SDN vs traditional networks

Reference:
Diego Kreutz, Fernando M. V. Ramos, Paulo Verissimo, Christian Esteve Rothenberg, Siamak Azodolmolky, Steve Uhlig,
Software-Defined Networking: A Comprehensive Survey.

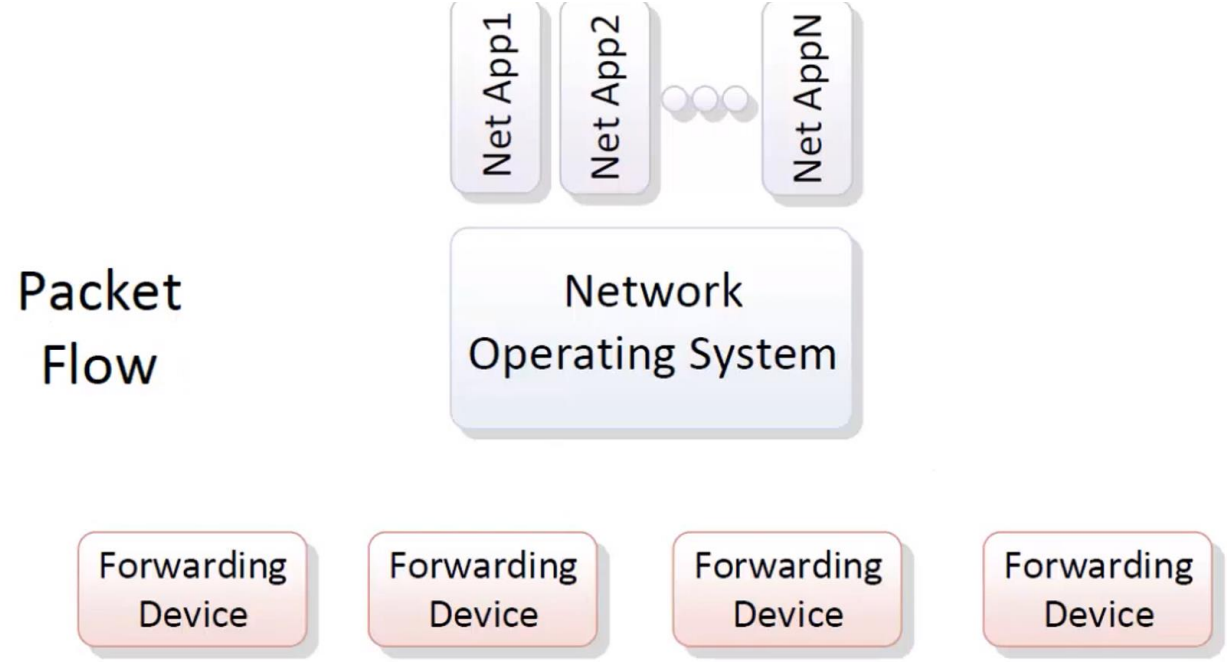
APPLICATION OF SDN

-Traffic Engineering
-Security
-QoS
-Routing
-Switching
-Virtualization
-Monitoring
-Load Balancing
-New Innovations ??

OS MODEL

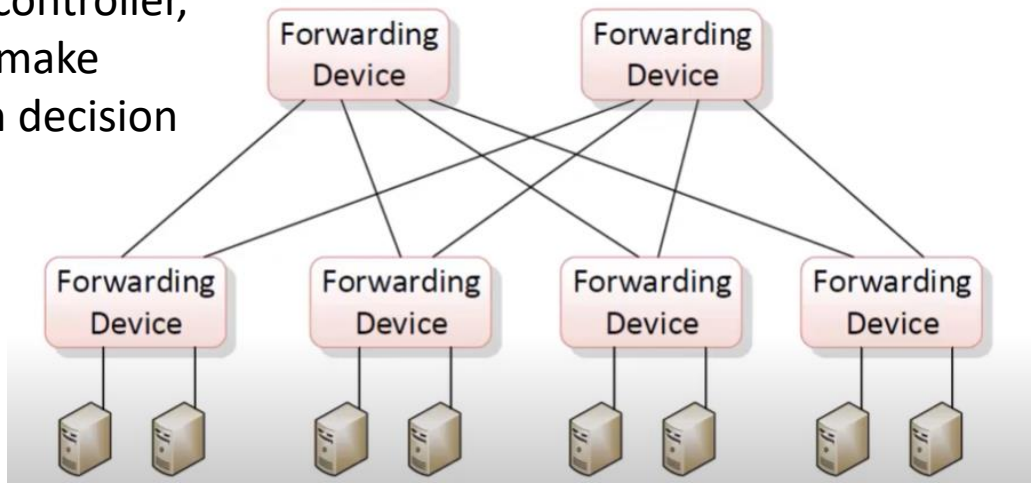


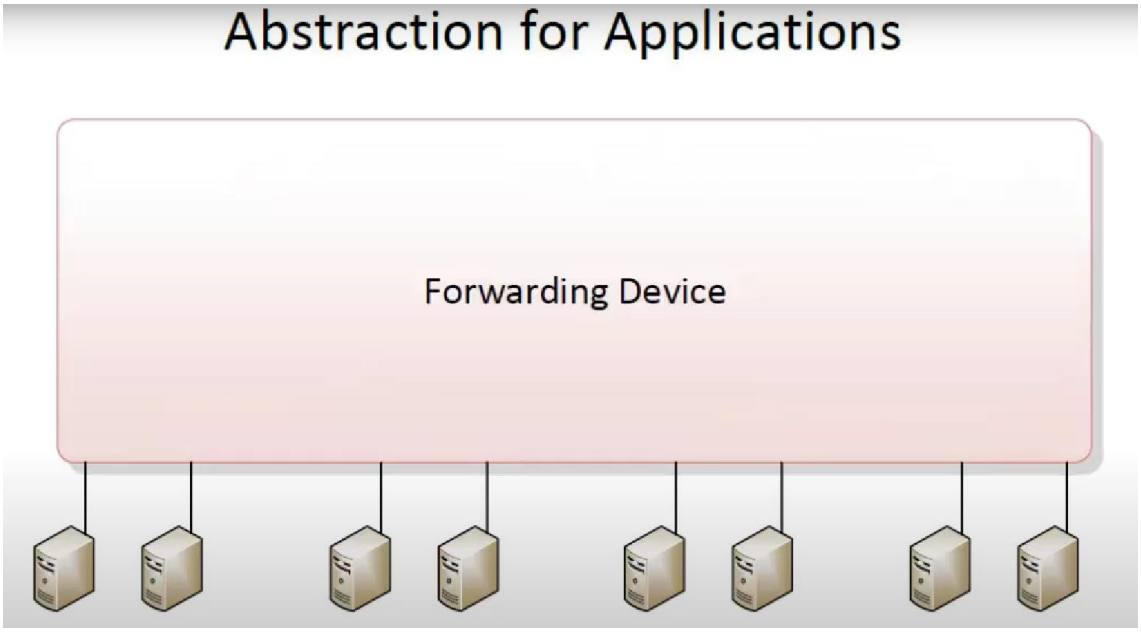
SDN MODEL



SDN MODEL

Without controller, they will make their own decision





With SDN, they will have central controller who has knowledge of whole topology

SDN CONTROLLER / CONTROL PLANE COMPONENTS

An SDN Controller is the central component of the Software-Defined Networking (SDN) architecture. It manages and programs network devices (like switches and routers) by providing a global network view and central control. It communicates with the data plane (switches) using protocols such as OpenFlow, and provides APIs for applications.

MAIN SERVICES OF SDN

⭐ Topology Service

Maintains an up-to-date map of the network topology. Discovers network devices (switches, routers, links). Helps the controller understand how switches are interconnected. Supports efficient path computation and network visualization.

➔ Example: The controller detects that Switch A is connected to Switch B and learns link bandwidth.

📚 Inventory Service

Keeps track of all network devices (switches, routers, hosts).
Stores metadata such as device types, port information, capabilities, etc.
Acts as a central registry of devices under control.

➔ Example: The controller knows that Switch ID=1 has 24 ports and supports OpenFlow 1.5.

MAIN SERVICES OF SDN

📊 Statistics Service

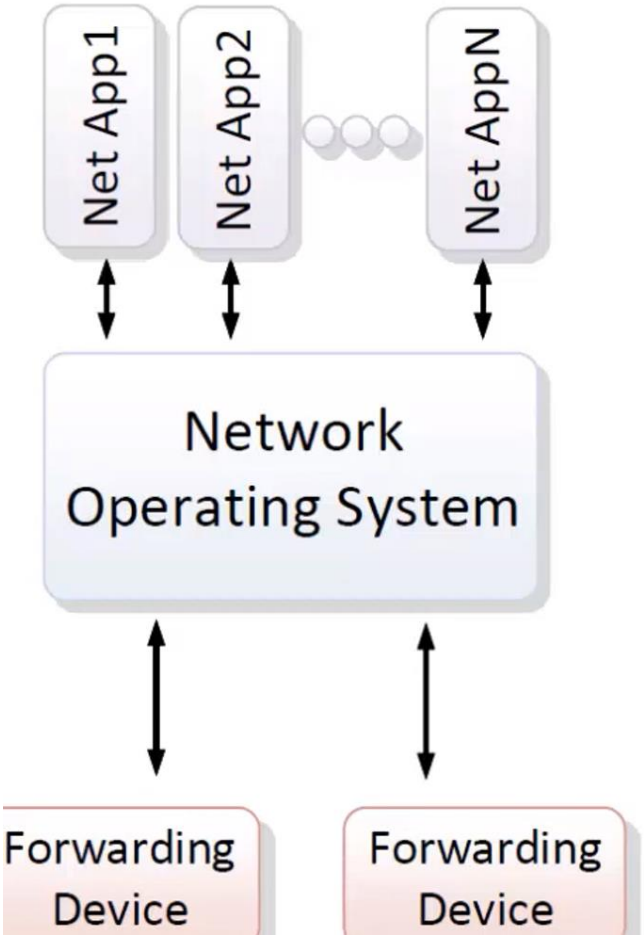
Collects real-time or periodic statistics from network devices. Monitors traffic counters, flow stats, port utilization, packet drops, errors. Enables traffic engineering decisions and performance monitoring.

➔ Example: The controller pulls port statistics every 5 seconds to track bandwidth usage and detect congestion.

🏠 Host Tracking Service

Tracks end-host devices (servers, clients) connected to the network. Maps IP/MAC addresses to switch ports. Helps in making forwarding decisions by knowing host locations.

➔ Example: Host with IP 192.168.1.10 is attached to Switch 3, Port 5.



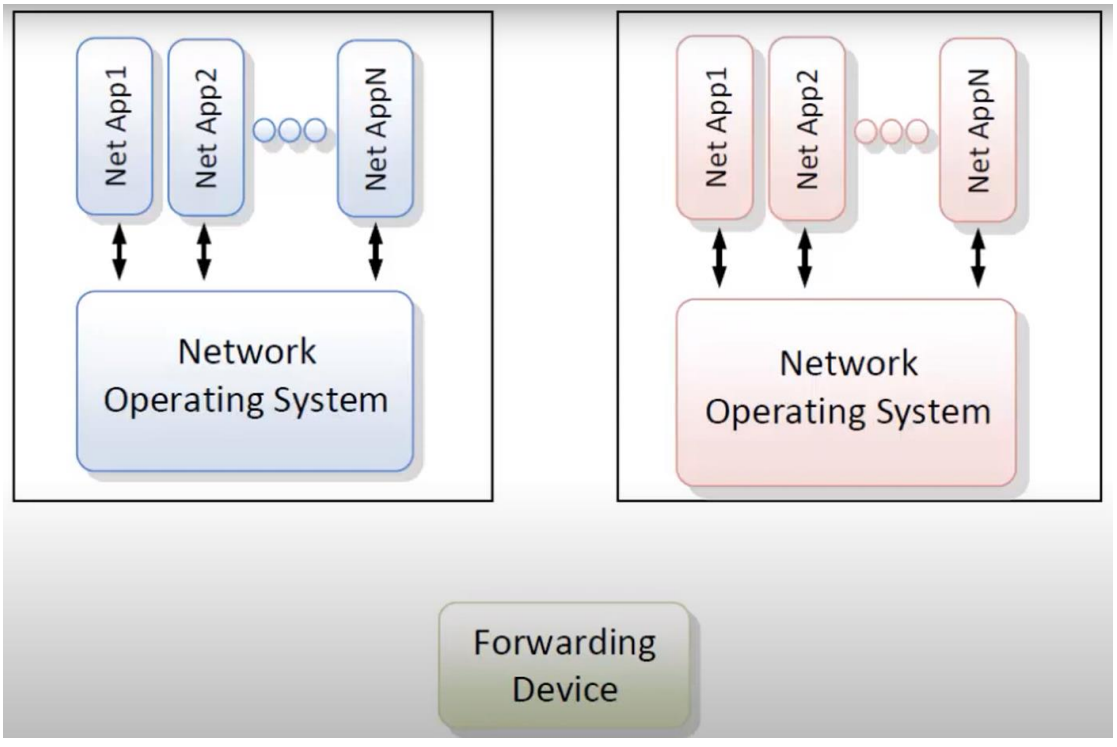
Network Applications

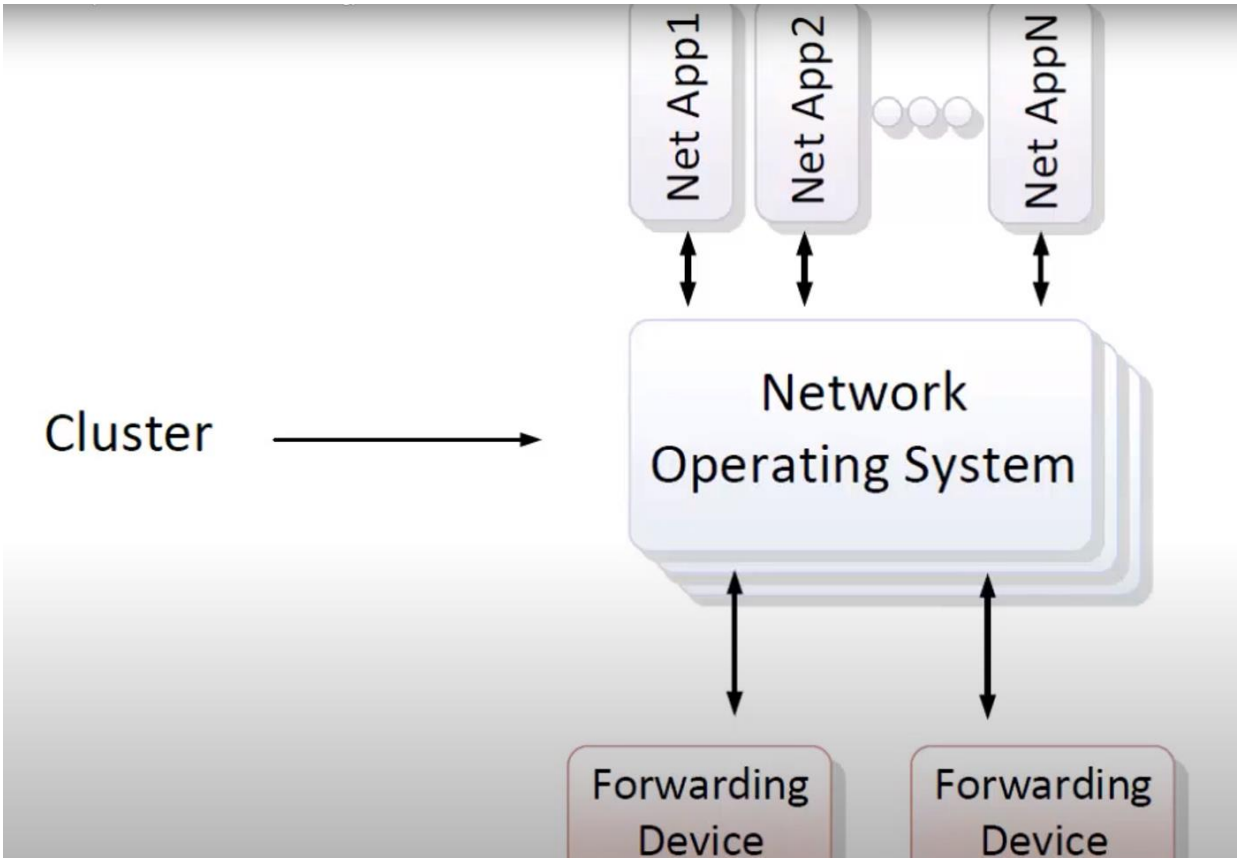
Application Interfaces
- Java API
- Northbound (e.g. RESTConf)

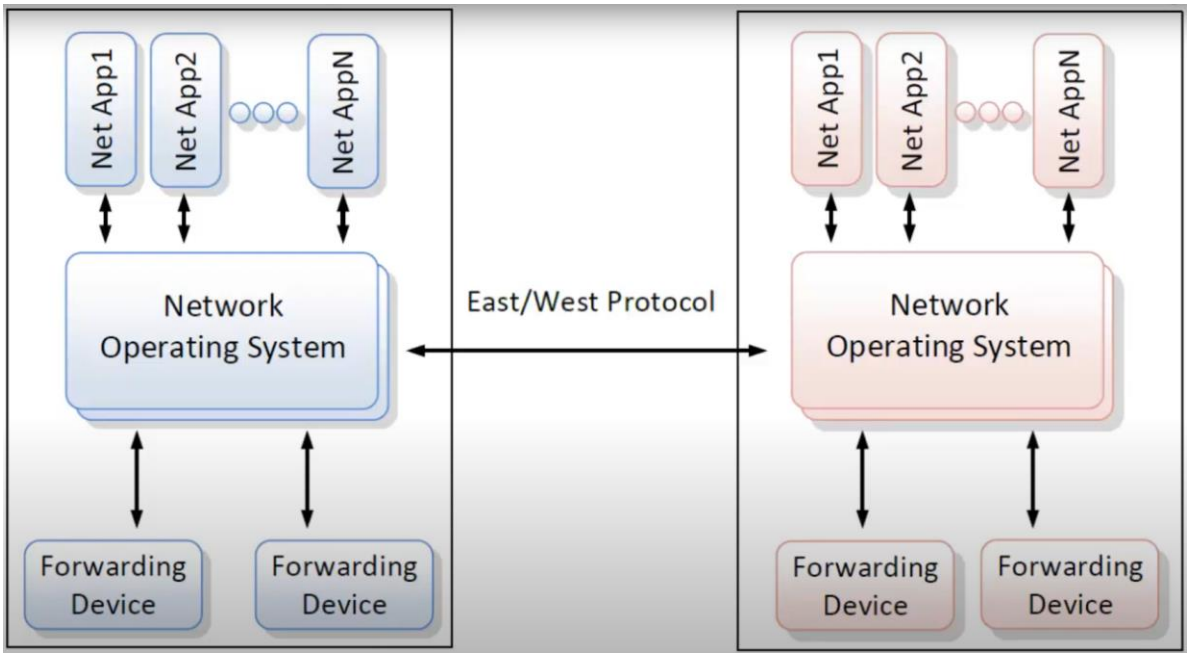
SDN Controller/Control Plane
- Topology Service
- Inventory Service
- Statistics Service
- Host Tracking

SouthBound Interface
- OpenFlow
- OVSDB
- NETCONF
- SNMP

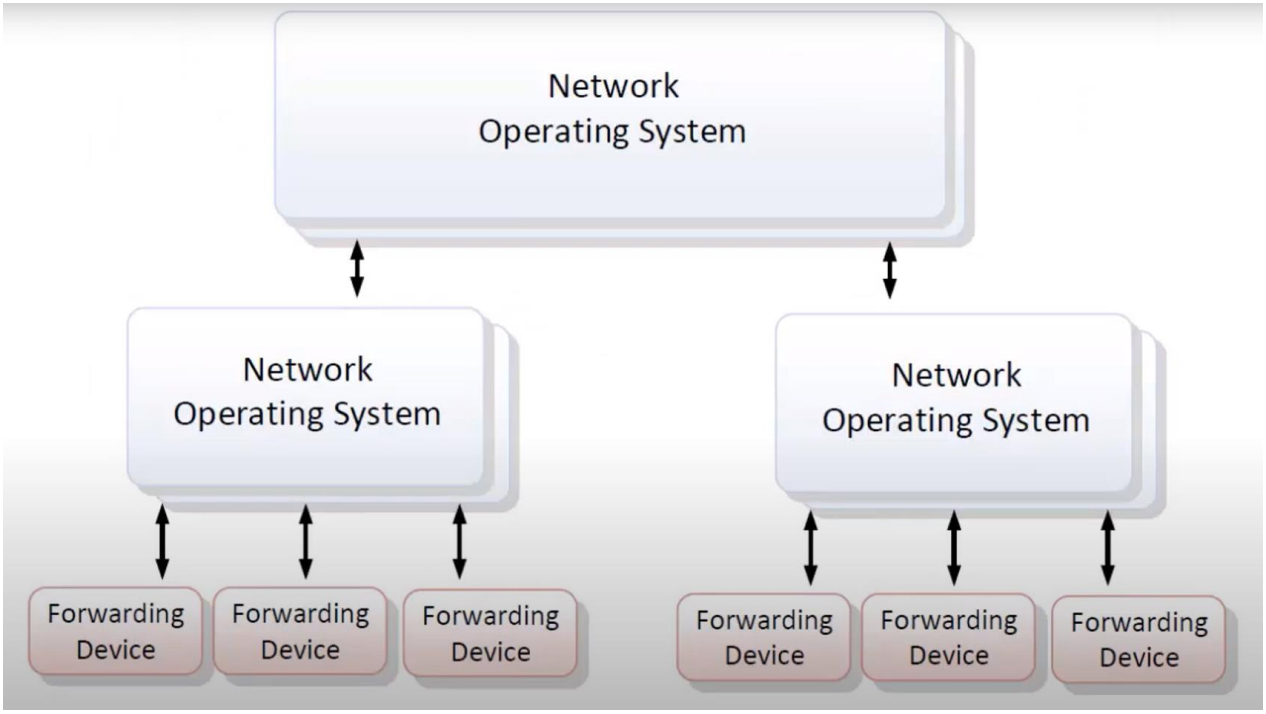
Forwarding Devices/ Data Plane

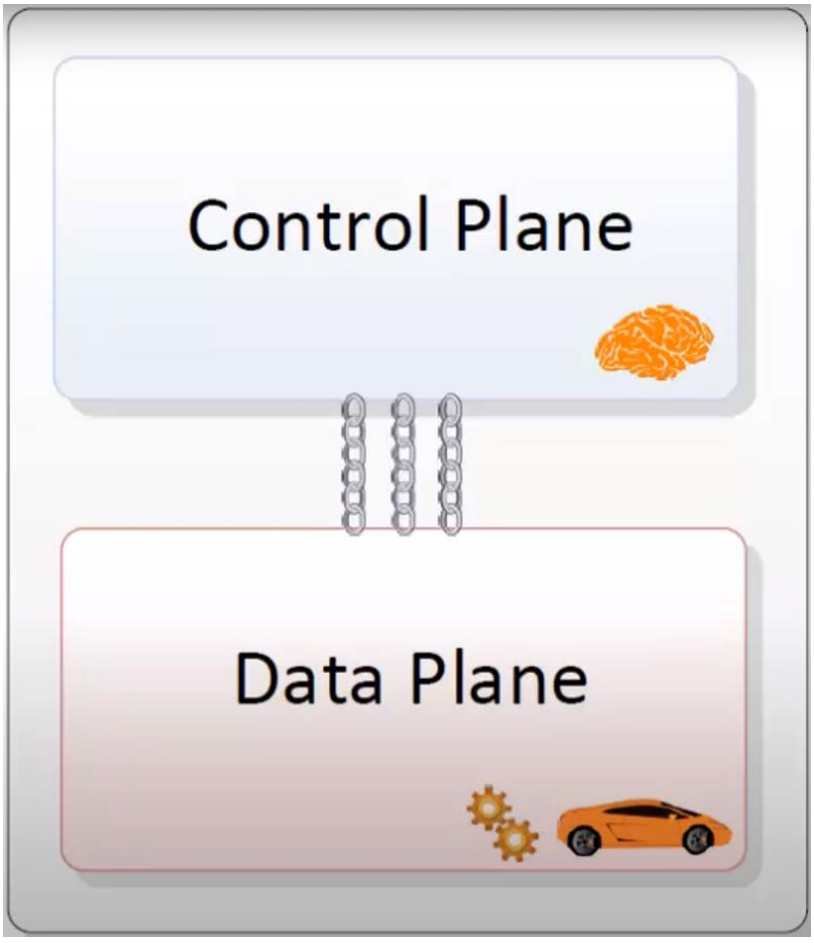






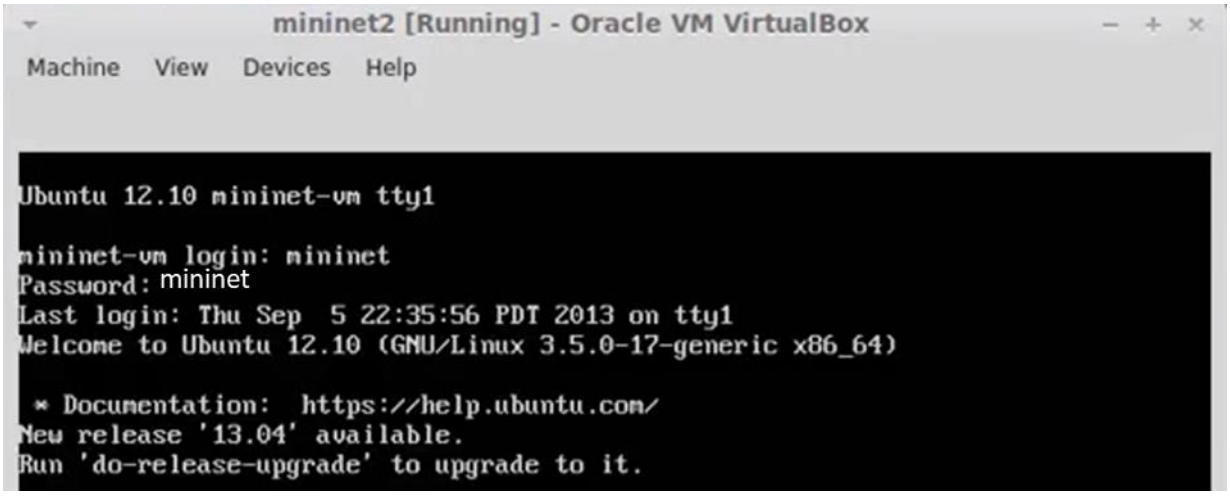
SDN into different regions and they can communicate using East/West Protocol.

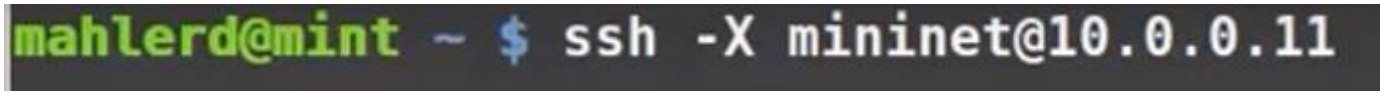

Network
Operating System
Network
Operating System
Network
Operating System
Forwarding
Device
Forwarding
Device
Forwarding
Device
Forwarding
Device
Forwarding
Device
Forwarding
Device


Control Plane
Data Plane

MININET (mininet.org)

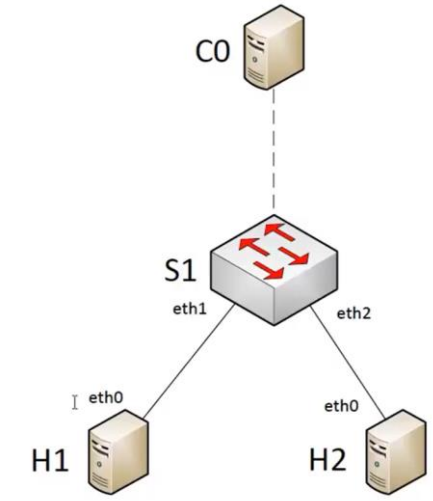
- Network emulation software that allows you to launch a virtual network with switches, hosts and an SDN controller all with a single command
- Great way to start learning about SDN and OpenFlow as well as test SDN controllers and SDN applications
- Go to mininet.org for full details, installation instructions, download a VM, tutorials, FAQs and to join the mailing list.





MININET (mininet.org)

```
mininet@mininet-vm:~$ sudo mn
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
*** Starting 1 switches
s1
*** Starting CLI:
```




```
mininet> help

Documented commands (type help <topic>):
========================================
EOF    exit   intfs     link   noecho       pingpair      quit   time
dpctl  gterm  iperf     net    pingall      pingpairfull  sh     xterm
dump   help   iperfudp  nodes  pingallfull  py            source
```

```
mininet> nodes
available nodes are:
h1 h2 c0 s1
```

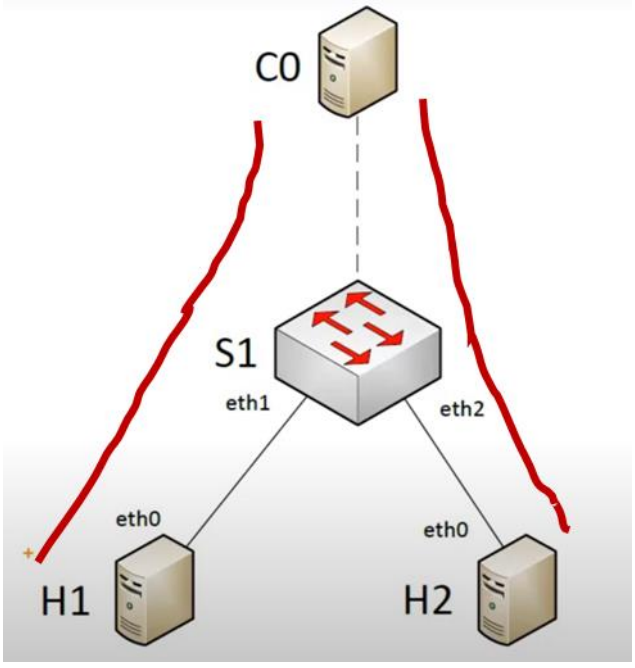
MININET (mininet.org)

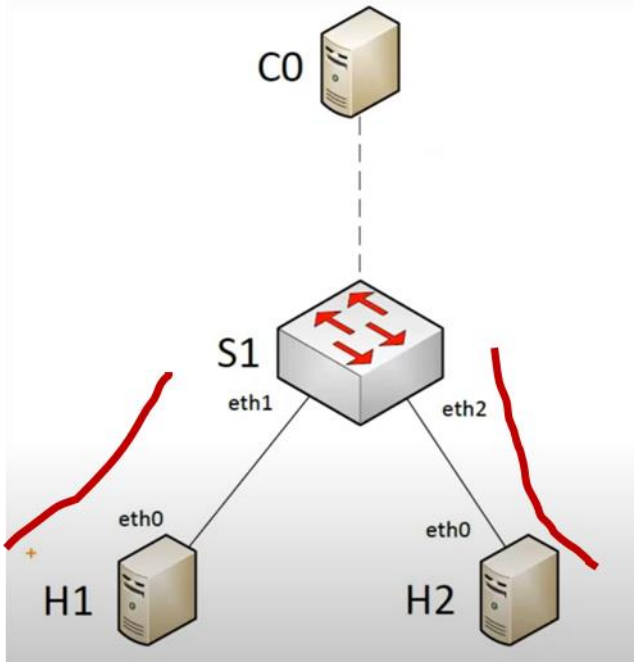
```
mininet> dump
<OVSController c0: 127.0.0.1:6633 pid=3431>
<OVSSwitch s1: 127.0.0.1:6634 lo:127.0.0.1,s1-eth1:None,s1-eth2:None pid=3443>
<Host h1: h1-eth0:10.0.0.1 pid=3439>
<Host h2: h2-eth0:10.0.0.2 pid=3440>
```

```
mininet> net
c0
s1 lo:  s1-eth1:h1-eth0 s1-eth2:h2-eth0
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
```

```
mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_req=1 ttl=64 time=6.31 ms
64 bytes from 10.0.0.2: icmp_req=2 ttl=64 time=0.250 ms
64 bytes from 10.0.0.2: icmp_req=3 ttl=64 time=0.089 ms
64 bytes from 10.0.0.2: icmp_req=4 ttl=64 time=0.108 ms
^C
--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 2999ms
rtt min/avg/max/mdev = 0.089/1.689/6.311/2.669 ms
```

PACKET FLOW





1st Packet goes via controller (extra latency)

2nd – last packet goes via S1

```
h2 h2-eth0:s1-eth2
mininet> dump
<OVSController c0: 127.0.0.1:
<OVSSwitch s1: 127.0.0.1:6634
<Host h1: h1-eth0:10.0.0.1 pi
<Host h2: h2-eth0:10.0.0.2 pi
mininet> net
c0
s1 lo:  s1-eth1:h1-eth0 s1-et
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(8
64 bytes from 10.0.0.2: icmp_
64 bytes from 10.0.0.2: icmp_
64 bytes from 10.0.0.2: icmp_
64 bytes from 10.0.0.2: icmp_
^C
--- 10.0.0.2 ping statistics
4 packets transmitted, 4 rece
rtt min/avg/max/mdev = 0.120/0.246/0.373/0.124 ms
mininet> xterm h1
```

```
Node: h1 (on mininet-vm)
root@mininet-vm:~#
```

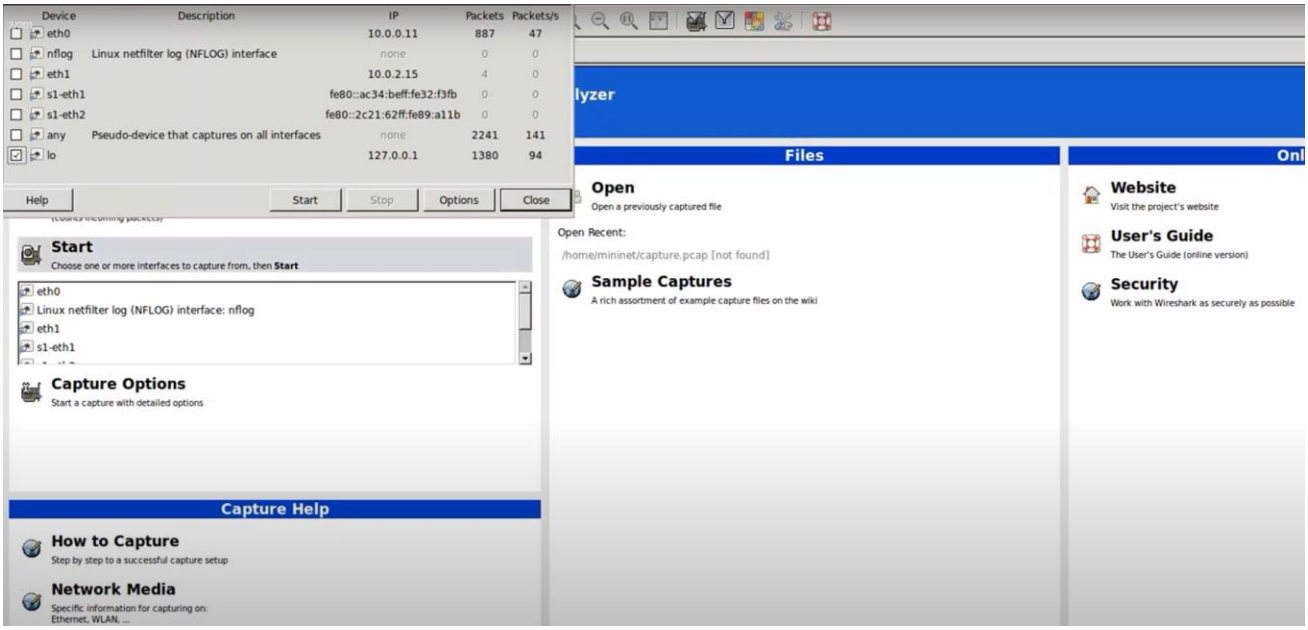
```
root@mininet-vm:~# ping -c3 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_req=1 ttl=64 time=2.14 ms
64 bytes from 10.0.0.2: icmp_req=2 ttl=64 time=0.351 ms
64 bytes from 10.0.0.2: icmp_req=3 ttl=64 time=0.107 ms

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2000ms
rtt min/avg/max/mdev = 0.107/0.866/2.142/0.907 ms
```

```
root@mininet-vm:~# tcpdump -n
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on h1-eth0, link-type EN10MB (Ethernet), capture size 65535 bytes
```

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (0/2 lost)
mininet> iperf
*** Iperf: testing TCP bandwidth between h1 and h2
waiting for iperf to start up...*** Results: ['214 Mbits/sec', '215 Mbits/sec']
mininet> iperfudp
```

sudo mn –c

mininet@mininet-vm:-$ sudo wireshark &

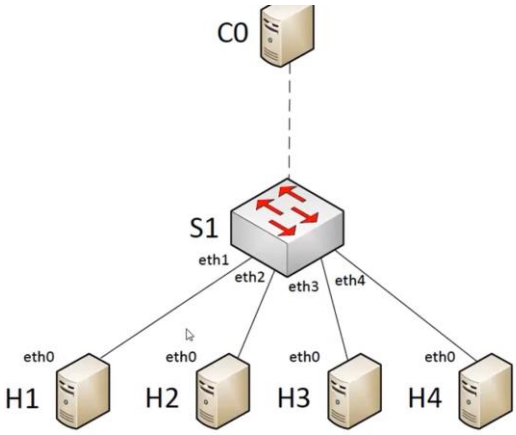


2131	2.396999000	127.0.0.1	127.0.0.1	OFP	74 Echo Request (SM) (8B)
2132	2.397512000	127.0.0.1	127.0.0.1	OFP	74 Echo Reply (SM) (8B)
4641	7.397663000	127.0.0.1	127.0.0.1	OFP	74 Echo Request (SM) (8B)
4642	7.398176000	127.0.0.1	127.0.0.1	OFP	74 Echo Reply (SM) (8B)
4926	12.39691400(	127.0.0.1	127.0.0.1	OFP	74 Echo Request (SM) (8B)
4927	12.39724500(	127.0.0.1	127.0.0.1	OFP	74 Echo Reply (SM) (8B)
5145	17.39699000(	127.0.0.1	127.0.0.1	OFP	74 Echo Request (SM) (8B)
5146	17.39735500(	127.0.0.1	127.0.0.1	OFP	74 Echo Reply (SM) (8B)

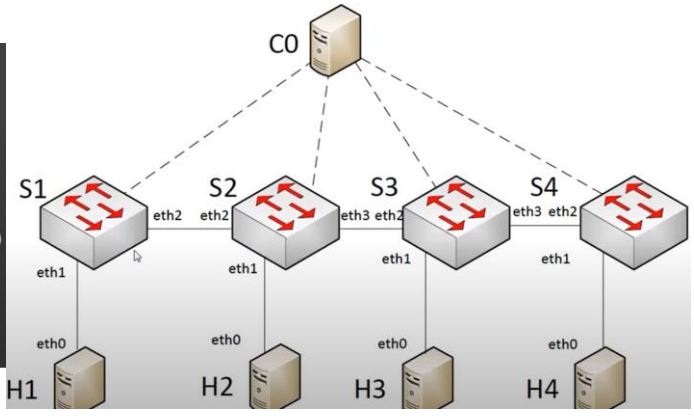
6510	36.36648000(	10.0.0.1	10.0.0.2	OFP+ICM	182 Packet In (AM) (BufID=271) (116B) => Echo (ping) request id=0x10f2, seq=1/256, ttl=64
6511	36.36708600(	127.0.0.1	127.0.0.1	OFP	146 Flow Mod (CSM) (80B)
6513	36.39117400(	10.0.0.2	10.0.0.1	OFP+ICM	182 Packet In (AM) (BufID=272) (116B) => Echo (ping) reply id=0x10f2, seq=1/256, ttl=64
6514	36.39175900(	127.0.0.1	127.0.0.1	OFP	146 Flow Mod (CSM) (80B)
6525	36.41504800(	10.0.0.2	10.0.0.1	OFP+ICM	182 Packet In (AM) (BufID=273) (116B) => Echo (ping) request id=0x10f3, seq=1/256, ttl=64
6526	36.41549300(	127.0.0.1	127.0.0.1	OFP	146 Flow Mod (CSM) (80B)
6529	36.43632400(	10.0.0.1	10.0.0.2	OFP+ICM	182 Packet In (AM) (BufID=274) (116B) => Echo (ping) reply id=0x10f3, seq=1/256, ttl=64

UNDERSTAND TOPOLOGY

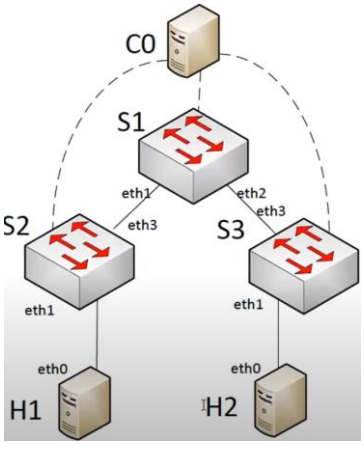
```
mininet@mininet-vm:~$ sudo mn --topo=single,4
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
*** Starting 1 switches
s1
```




```
mininet@mininet-vm:~$ sudo mn --topo=linear,4
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (s1, s2) (s2, s3) (s3, s4)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
*** Starting 4 switches
s1 s2 s3
```

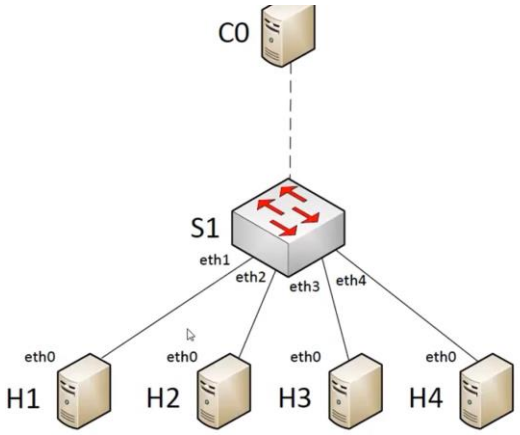



```
mininet@mininet-vm:~$ sudo mn --topo=tree,2,2
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3
*** Adding links:
(h1, s2) (h2, s2) (h3, s3) (h4, s3) (s1, s2) (s1, s3)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
*** Starting 3 switches
s1 s2 s3
```

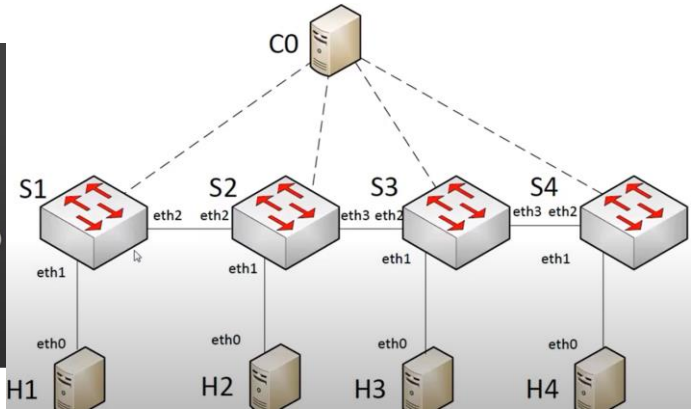


UNDERSTAND TOPOLOGY

```
mininet@mininet-vm:~$ sudo mn --topo=single,4
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
*** Starting 1 switches
s1
```




```
mininet@mininet-vm:~$ sudo mn --topo=linear,4
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (s1, s2) (s2, s3) (s3, s4)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
*** Starting 4 switches
s1 s2 s3
```




```
mininet@mininet-vm:~$ sudo mn --topo=tree,2,2
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3
*** Adding links:
(h1, s2) (h2, s2) (h3, s3) (h4, s3) (s1, s2) (s1, s3)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
*** Starting 3 switches
s1 s2 s3
```